

Learn React + Next.js by Building a Snooker Fantasy League

Learn React + Next.js by Building a Snooker Fantasy League

A structured, beginner-friendly course that teaches you React from scratch by **rebuilding** a real, working fantasy-sports app — one component at a time, in the order a senior engineer would actually build it. Designed to take you from zero to interview-ready.

Who this course is for

You. Specifically, you if you:

- Have written a tiny bit of JavaScript or another language (Python, Java, etc.) but **never built a real React app**.
- Want to understand **why** React is structured the way it is — not just memorize syntax.
- Want to prepare for **React interviews** by building something you can show off and explain.
- Want to get good at **vibe coding** — directing an AI to produce clean, maintainable code instead of accepting whatever it spits out.

If you've already built three React apps and just want a quick reference, the single-file course at `SNOOKER_FANTASY_LEAGUE_COURSE.md` (preserved untouched in this folder) is a denser, faster read. This expanded multi-chapter version is the one to read if you actually want to **learn the craft**.

What you'll build

By the end of this course you will have built — by hand, line by line, with full understanding — a complete Next.js + TypeScript application:

- A **standings table** ranking 8 fantasy teams by points
- A **matches view** with cards for every match across 5 rounds ($32 \times 16 \times 8 \times 4 \times 1 = 31$ matches)
- A **predictions matrix** comparing 8 teams' picks side-by-side
- A **players directory** with deep per-player drill-downs (32 player profiles)
- An **analytics dashboard** with five Recharts visualizations

The finished version of all of this lives at `components/`, `lib/`, and `app/` in the project root. **You'll rebuild it from scratch as you learn.** The course tells you exactly what to type, why to type it, and what each line does.

What you'll learn (the React inventory)

By the time you finish Chapter 7, you will have used and **understood** every one of these React concepts:

Concept	Where you'll meet it
Functional components	Chapter 3
Props & prop types	Chapter 3
<code>useState</code>	Chapter 4
<code>useMemo</code>	Chapter 4
Lifting state up	Chapter 4
Conditional rendering	Chapter 3, 4
Lists & keys	Chapter 3
Event handlers	Chapter 4
Component composition	Chapter 5
Prop drilling vs Context	Chapter 5
Controlled components	Chapter 4
TypeScript with React	Throughout
Inline styles vs Tailwind vs CSS	Chapter 3
Pure functions in UI code	Chapter 1
Data transforms for charts	Chapter 6

Concept	Where you'll meet it
Server vs client components (Next.js App Router)	Chapter 2, 7
useState inside a child vs at the top	Chapter 6
Re-renders, references, and identity	Chapter 4

You'll also pick up the **mental models** that experienced React developers carry around — when to split a component, when to memoize, what state lives where, why prop drilling is fine for three levels and a problem at six. These don't show up in tutorials, but they show up in every senior interview.

How the course is structured

Seven chapters. Each chapter has three lessons. Each lesson is around 1,500–2,500 words and ends with a **CHECK YOURSELF** box of questions you should be able to answer before moving on.

```

course/
├── README.md                                ← you are here
├── SNOOKER_FANTASY_LEAGUE_COURSE.md        ← original single-
file version (reference)
├──
├── chapter-01-foundations/                  ← UNDERSTAND THE PROBLEM
│   ├── 01-the-problem.md
│   ├── 02-data-modeling.md
│   └── 03-pure-functions-and-scoring.md
├──
├── chapter-02-project-setup/                ← FROM EMPTY FOLDER TO RUNNING DEV SERV
│   ├── 01-why-nextjs-typescript.md
│   ├── 02-scaffolding-the-project.md
│   └── 03-creating-data-files.md
├──
├── chapter-03-first-component/              ← YOUR FIRST REACT COMPONENT
│   ├── 01-anatomy-of-a-component.md
│   ├── 02-rendering-the-standings.md
│   └── 03-progressive-enhancement.md

```

└─ chapter-04-state-and-hooks/	← INTERACTIVITY & SHARED STATE
└─ 01-useState-mental-model.md	
└─ 02-the-orchestrator-pattern.md	
└─ 03-useMemo-and-derived-data.md	
└─ chapter-05-composition/	← BIG COMPONENTS, SMALL COMPONENTS
└─ 01-when-to-split-components.md	
└─ 02-prop-drilling-vs-context.md	
└─ 03-building-player-detail.md	
└─ chapter-06-data-visualization/	← CHARTS WITH RECHARTS
└─ 01-recharts-fundamentals.md	
└─ 02-reshaping-data.md	
└─ 03-pivots-and-internal-state.md	
└─ chapter-07-mastery-and-interviews/	← INTERVIEW PREP & VIBE CODING
└─ 01-production-polish.md	
└─ 02-deployment-and-testing.md	
└─ 03-interview-narrative.md	

Each chapter folder also has its own README.md with a chapter-level overview, a list of new concepts introduced, and a list of files you'll have created by the end of the chapter.

How to use this course

The recommended workflow

1. **Read the lesson** in your IDE or browser.
2. **Open a second editor window** showing the *finished* version of whatever file the lesson is teaching (in components/, lib/, etc.). Use it as the answer key.
3. **Type the code yourself** as you go. Don't copy-paste. Typing is how the patterns become muscle memory.
4. **Run npm run dev** at the end of each lesson and look at the result in the browser. If the screen looks wrong, you skipped something.
5. **Answer the CHECK YOURSELF questions out loud** (or in writing) before moving on. They're not optional. They're the difference between *reading* React and *knowing* Re-

act.

6. **Commit at the end of every lesson** with a message like `Chapter 3 Lesson 2: render the standings table`. Building a real git history is part of the point.

Pace yourself

The full course is about 35,000 words plus typing the code. Realistically:

- **One lesson per evening (1–2 hrs)** = comfortable pace, ~3 weeks total.
- **One chapter per weekend** = aggressive but doable, ~3 weekends.
- **Binge** = don't. You'll forget. Spaced repetition wins.

Don't skip Chapter 1

Chapter 1 has zero React in it. It's about **the problem** and **the data model**. Beginners always want to skip to "the React part." Don't. Half of senior interview questions are *"How would you model X?"* and *"What state is owned where?"* — and those are answered before any JSX gets typed. Chapter 1 is where you build the muscle for that.

Reading vs building

Each lesson has two layers:

- **Concepts** — the *why*. Why do components return JSX? Why is `useState` a hook? Why does this data shape work for this chart?
- **Code** — the *what*. The exact lines you'll type, copy-pasted from the real codebase.

Both matter. If you only read the concepts, you'll be a React philosopher who can't ship. If you only type the code, you'll be a React typist who can't debug. The course alternates deliberately.

What's in the rest of the repo

Beyond the course folder, the repo contains the **finished application** so you can compare your work to the answer key at any point:

```
e:\reverse-understanding\  
└─ _source/
```

```

|   └─ SnookerFantasyLeague.jsx    ← the original 1,654-line single-
file build
|
|                                   (preserved verbatim, the artifact this course
|                                   was reverse-engineered from)
|
|   └─ _briefs/
|     └─ PROMPT_1_NEXTJS_REFACTOR.md ← the spec that generated the refactor
|       └─ PROMPT_2_REVERSE_ENGINEERING_COURSE.md
|         └─ ← the spec that generated this course
|
|   └─ app/                        ← Next.js App Router entry points
|   └─ components/                 ← all the React components, organized by feature
|     └─ SnookerFantasyLeague.tsx ← the top-level orchestrator
|     └─ tabs/                     ← StandingsTab, MatchesTab, PredictionsTab, ...
|       └─ standings/              ← LeagueTable, StatCard, FormBadge
|       └─ matches/                ← MatchesList, PlayerLine, RoundButton
|       └─ predictions/            ← PredictionMatrix, TeamCardView, PicksList
|       └─ players/                ← PlayerCard, PlayerDetail, MatchPickAnalysis
|       └─ analytics/              ← LeagueAgreementChart, ChartCard, InsightCard
|   └─ lib/                        ← pure data + logic (no React)
|     └─ types.ts                  ← TypeScript interfaces
|     └─ matches.ts                ← all 31 matches
|     └─ teams.ts                  ← all 8 fantasy teams + their picks
|     └─ players.ts                ← player metadata
|     └─ scoring.ts                ← scorePick + calculateTeamScores
|     └─ constants.ts              ← shared style helpers + ball colors
|   └─ package.json
|   └─ tsconfig.json
|   └─ tailwind.config.ts
|   └─ next.config.mjs

```

You'll re-create most of these files yourself by following the lessons. The originals stay there as your reference.

Prerequisites

You **need**:

- Node.js 20+ (for Next.js 14)

- npm (comes with Node)
- A code editor (VS Code, Cursor, Windsurf — anything)
- A browser
- ~15–20 hours of focused time

You **don't need**:

- Prior React experience
- A computer-science degree
- Mac or Linux (this course was written on Windows; PowerShell commands are noted explicitly)
- Any paid services

Conventions used in this course

- **backticks** for variable, function, file, and component names.
- **Code blocks** with a language tag (````tsx`, ````ts`, ````bash`, ````json`).
- **CHECK YOURSELF boxes** at the end of each lesson. Don't skip them.
- **Real code only.** Every code snippet in this course is taken from the actual finished app in this repo. There are no toy examples or simplified pretend-code.
- **TypeScript everywhere.** If you've never touched TS, don't worry — the lessons explain types as they appear.

A note on AI / vibe coding

The original build of this app was done with AI assistance (“vibe coding”). This course teaches you both the **React fundamentals** and the **prompting craft** that produced the codebase. Chapter 7 is dedicated to making you a better vibe coder: when an LLM gives you React, what do you accept and what do you reject? When do you ask for “extract this into a component” versus “rewrite this”?

Interviewers in 2026 don't ask “Do you use AI?” — they ask “**How** do you use AI?” If you can articulate why a particular component should be split, why a particular piece of state lives where it does, why a particular memoization is or isn't worth it — you sound like a senior, regardless of whether you typed the code or prompted it.

Ready?

Start at [Chapter 1, Lesson 1: The Problem](#).

You won't write a single line of JavaScript in Chapter 1. By the end of Chapter 7 you'll have built a complete Next.js application from scratch and you'll be able to explain every architectural decision in it. **That's how you pass interviews.**